# Hybrid Path Tracking Control for Autonomous Trucks: Integrating Pure Pursuit and Deep Reinforcement Learning With Adaptive Look-Ahead Mechanism

Zhixuan Han, Peng Chen, *Member, IEEE*, Bin Zhou, and Guizhen Yu

***Abstract*— Path tracking control is essential for ensuring the safe and efficient operation of autonomous trucks, but traditional methods often struggle with nonlinear vehicle dynamics. While deep reinforcement learning (DRL) approaches are model-free, they may lack the stability and interpretability required for reliable deployment. This study presents a hybrid control framework that combines Pure Pursuit (PP) with Proximal Policy Optimization (PPO) to enhance tracking accuracy and robustness. PP provides baseline stability and interpretability, while PPO refines control actions by optimizing policy gradients, ensuring better adaptability to nonlinear dynamics and complex driving conditions. An adaptive look-ahead mechanism, responsive to speed and curvature, dynamically adjusts preview distances using PPO-generated coefficients, facilitating early corrections during high-speed turns and enabling greater precision on sharp curves. A fusion training method, leveraging high-reward initialization and a decreasing learning rate, supports efficient exploration and stable convergence. The approach was validated in a high-fidelity simulation environment using *PreScan*, *Simulink*, and *ROS*, along with real-world experiments on a proportionally scaled intelligent vehicle chassis, demonstrating notable improvements in path tracking accuracy and robustness across varied path profiles.**

***Index Terms*— Path tracking control, deep reinforcement learning (DRL), adaptive look-ahead mechanism, high-speed turns.**

## I. Introduction

AUTONOMOUS trucks play a crucial role in modern logistics systems, where enhancing operational efficiency and safety is paramount [1], [2]. Path tracking control is a fundamental for realizing safe and efficient autonomous driving, as it ensures the vehicle follows the planned path accurately while maintaining stability [3], [4], [5]. This control task is especially challenging due to the diverse and complex environments autonomous trucks must navigate, such as sharp turns, continuous curves, and narrow lanes, as illustrated in Fig. 1 [6].

Received 3 August 2024; revised 19 November 2024; accepted 11 January 2025. Date of publication 27 January 2025; date of current version 5 May 2025. This work was supported in part by the National Key Technologies Research and Development Program of China under Grant 2022YFB4703700 and in part by the National Natural Science Foundation of China under Grant 52272327. The Associate Editor for this article was Y. Wiseman. *(Corresponding author: Peng Chen.)*

The authors are with the Key Laboratory of Autonomous Transportation Technology for Special Vehicles, Ministry of Industry and Information Technology, School of Transportation Science and Engineering, Beihang University, Beijing 100191, China (e-mail: buaa_hzx@buaa.edu.cn; cpeng@buaa.edu.cn; binzhou@buaa.edu.cn; yugz@buaa.edu.cn).

Digital Object Identifier 10.1109/TITS.2025.3530507

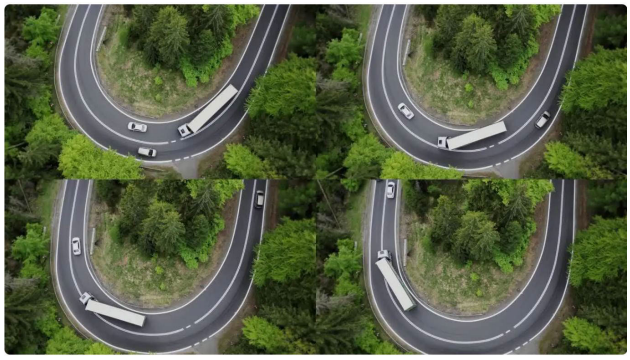

Fig. 1. Challenging environments (https://www.vjshi.com/watch/24986521.html).

Traditional path tracking control algorithms typically depend on system models and manually configured control strategies. For instance, model-based approaches typically utilize simplified kinematic models, which may not capture the full dynamics of the vehicle, especially under complex conditions. The use of these simplified models can degrade performance in precise modeling and feature design under varying conditions. On the other hand, model-free approaches, such as Proportional-Integral-Derivative (PID) controllers, rely on tracking error feedback to generate control commands without explicitly using a vehicle model. Although PID controllers and similar strategies are simpler and more adaptable to different conditions, they can struggle to cope with the dynamic and unpredictable environments encountered by autonomous trucks, such as varying speeds and sharp curves. Both types of traditional algorithms face significant challenges in maintaining performance and stability in complex scenarios.

In contrast, Deep Reinforcement Learning (DRL) has shown considerable promise in path tracking control due to its end-to-end learning capability, nonlinear modeling, and adaptive learning [7], [8]. DRL's end-to-end learning capability enables it to derive control strategies directly from sensor data, making it well-suited for complex dynamic environments [9]. Through interaction with the environment, DRL achieves autonomous learning and adaptability to new conditions, thereby enhancing system robustness. These characteristics address the limitations of traditional control algorithms in path tracking control, opening new avenues for improving the performance of

1558-0016 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: American University of Beirut. Downloaded on January 28,2026 at 19:35:21 UTC from IEEE Xplore. Restrictions apply.

autonomous vehicle path tracking. However, a major drawback of DRL is its low data efficiency and limited interpretability, which can lead to long training times and unexpected control outcomes [10].

Given the limitations of both traditional control methods and standalone DRL approaches, this study proposes a hybrid truck path tracking controller that combines the Pure Pursuit (PP) algorithm and DRL. The PP algorithm, which relies on a simple kinematic model, serves as the primary control component. However, its limitations in capturing the full vehicle dynamics can lead to inaccuracies, especially in complex scenarios. To address this, we integrate DRL, which compensates for the errors generated by the PP algorithm by utilizing its ability to model complex vehicle dynamics. In this hybrid controller, the turning angle dictated by the PP algorithm serves as the primary control component, while the turning angle output by DRL acts as a compensatory element to correct errors generated by the PP algorithm, thereby achieving more effective path tracking.

To validate this approach, we built a vehicle dynamics system that demonstrates how DRL can bridge the gap left by the kinematic model of the PP algorithm. Our results confirm that DRL successfully compensates for the limitations of the PP algorithm, enhancing performance and robustness in scenarios involving sharp turns, varying speeds, and complex trajectories. The main contributions of this study are as follows:

- **Hybrid Control Framework:** We propose a novel hybrid framework that integrates Pure Pursuit (PP) and Proximal Policy Optimization (PPO), utilizing PP's inherent stability and interpretability while enabling PPO to adaptively refine control actions for complex vehicle dynamics.
- **Adaptive Look-Ahead Mechanism:** An adaptive look-ahead mechanism is introduced to dynamically adjust preview distances based on vehicle's speed and road curvature. Supported by PPO-generated coefficients, this mechanism enables real-time adjustments during high-speed turns and improved tracking accuracy.
- **Fusion Training Method:** A fusion training method is implemented that combines high-reward initialization with a decreasing learning rate to promote efficient exploration and stable convergence, ultimately enhancing the training effectiveness of deep reinforcement learning in path tracking.
- **High-Fidelity Simulation and Real-World Validation:** The proposed approach is validated in a high-fidelity simulation environment constructed with *PreScan*, *Simulink*, and *ROS*, and further corroborated through real-world experiments using a proportionally scaled intelligent vehicle chassis, demonstrating significant improvements in path tracking accuracy and robustness across varied path profiles.

## II. Related Works

Path tracking is a fundamental task in the development of autonomous vehicles, ensuring that the vehicle accurately follows a desired trajectory while navigating through dynamic and uncertain environments. Traditional control approaches and newer machine learning-based methods have been widely investigated to improve tracking accuracy, robustness, and computational efficiency. In this section, we review the primary categories of path tracking methods, including model-free control methods, optimal control methods, geometric methods, and machine learning-based approaches, highlighting their strengths and limitations.

### A. Model-free Control Methods

Model-free control methods does not rely on explicit vehicle dynamics model but instead focus on controlling the system based on tracking errors. Among the most widely used techniques in this category is the Proportional-Integral-Derivative (PID) controller, which adjusts control actions based on the vehicle's deviation from the desired path. Although PID controllers are simple, easy to implement, and computationally efficient, they often struggle with nonlinear systems and complex, dynamic environments [11], [12]. Tuning the PID parameters for optimal performance remains a significant challenge, especially in real-time applications with varying vehicle dynamics.

Recent studies, such as that of Shi and Zhang [13], have sought to enhance PID controllers by optimizing parameters using techniques like the Grey Wolf Optimizer. While such approaches improve performance, they still face difficulties in handling large-scale nonlinearities and uncertainties in vehicle dynamics, particularly in unstructured environments. The inability to predict long-term system behavior limits the scalability of pure model-free control methods.

### B. Optimal Control Methods

Optimal control approaches, particularly Model Predictive Control (MPC), have become increasingly prominent in path tracking due to their ability to handle multi-variable systems with constraints by transforming the control problem into an optimization problem. Specifically, MPC uses a mathematical model to predict the system's behavior over a future time horizon and optimizes control inputs based on these predictions. At each control time step, MPC calculates the optimal control inputs by solving an optimization problem that takes into account the current system state and target requirements. This approach has exhibited significant potential in navigating the trade-offs between tracking accuracy and system limitations, particularly in environments marked by diverse road conditions and vehicle dynamics [14]. However, the dependence of MPC on solving an optimization problem at each time step may lead to significant computational costs, thereby constraining its real-time applicability, particularly in resource-constrained autonomous vehicles [15].

Several advancements have been proposed to mitigate the computational burden of MPC. For instance, Rokonuzzaman et al. [15] introduced a simplified MPC framework that reduces computational demands by utilizing less complex models, though this often sacrifices some accuracy and robustness. Despite these improvements, MPC remains less suitable for real-time path tracking in fast-paced environments with

Authorized licensed use limited to: American University of Beirut. Downloaded on January 28,2026 at 19:35:21 UTC from IEEE Xplore. Restrictions apply.

dynamic constraints, where computation times may become unpredictable and lead to instability in control [16].

### C. Geometric Methods

Geometric methods, such as the Pure Pursuit (PP) algorithm, are widely used in path tracking control due to their simplicity and effectiveness, particularly in scenarios with relatively constant vehicle dynamics. These methods focus on steering the vehicle along a desired path based on geometric principles, without considering the vehicle's internal dynamics. In path tracking applications, kinematic vehicle models are often employed to describe the vehicle's motion in terms of position, velocity, and acceleration, while neglecting the internal forces or dynamics. This simplified model is particularly useful in situations where the vehicle's internal dynamics are either less significant or unknown, offering a computationally efficient way to approximate motion [17].

The PP algorithm steers the vehicle towards a preview point ahead of its current position, adjusting the steering angle to minimize path-following errors. This approach works well in Ackermann steering vehicles and is effective in many path tracking scenarios [17], [18]. However, the performance of the PP algorithm is highly sensitive to the selection of the preview distance, which needs to be carefully tuned to account for varying vehicle speeds and road conditions.

To address this limitation, the Stanley method was introduced. The Stanley method dynamically adjusts the preview distance based on the vehicle's speed and heading error, improving its adaptability to changing driving conditions [19]. Despite these enhancements, both the PP and Stanley methods still face challenges in highly dynamic environments, where real-time feedback becomes essential for robust path tracking. In such cases, feedforward control alone may not be sufficient to ensure optimal performance.

### D. Machine Learning-Based Methods

Recent advancements in deep reinforcement learning (DRL) have shown significant potential in addressing the limitations of traditional path tracking methods. DRL's ability to learn directly from sensor data, bypassing the need for accurate system models, enables it to adapt to a wide range of dynamic and uncertain environments. DRL-based methods can autonomously derive complex control strategies that improve tracking performance, especially in environments where traditional approaches struggle with nonlinearities or unpredictable changes [7], [8].

For example, Moon et al. [21] proposed a DRL-based path tracking controller for drones, achieving highly accurate target tracking with low runtime costs. Similarly, Okafor et al. [22] combined DRL with PID control to design a hybrid controller for path tracking in mobile robots, demonstrating better performance compared to standalone PID controllers. DRL's ability to learn optimal policies from real-world data enables it to handle complex dynamic systems more effectively than classical methods [24].

However, the application of DRL to path tracking faces several challenges, including the need for large amounts of training data and the relatively slow convergence of learning processes [10]. Furthermore, the lack of interpretability in DRL models makes it difficult to ensure stability and predictability in real-world applications, which are crucial for autonomous driving systems. Recent work has attempted to address these issues by combining DRL with more traditional methods. For instance, Chen and Chen [20] integrated DRL with the PP algorithm, using DRL to adjust the steering commands generated by PP. Although this hybrid approach improves performance in some cases, it still faces limitations such as the absence of a mechanism to account for vehicle speed and path curvature, and its performance has yet to be validated in real-world settings.

### E. Hybrid Methods

In light of the limitations of both traditional and machine learning-based methods, hybrid approaches have been proposed to combine the strengths of both [23]. For example, the approach proposed in this study integrates the PP algorithm with Proximal Policy Optimization (PPO), a type of DRL, to achieve more robust and adaptable path tracking. In this hybrid framework, PP provides baseline stability and interpretability, while PPO refines control actions by optimizing policy gradients. This combination enhances the ability of the path tracking controller to handle complex driving conditions, including sharp turns and varying vehicle speeds, while maintaining the interpretability and stability of classical methods.

### F. Summary

In summary, while traditional model-based control methods such as PID, MPC, and PP have demonstrated effectiveness in various path tracking scenarios, they are often limited by their inability to handle highly dynamic and nonlinear systems in real-world environments. Machine learning-based methods, particularly DRL, offer significant promise in overcoming these limitations, but challenges such as data inefficiency and lack of interpretability remain. Hybrid approaches that combine the stability of traditional methods with the adaptability of machine learning represent a promising avenue for future research. The work presented in this study builds on this hybrid paradigm, aiming to achieve a more robust and adaptable solution for autonomous truck path tracking.

## III. Methodology

In this chapter, we introduce a novel approach to path tracking control that effectively combines traditional pure pursuit algorithms with advanced deep reinforcement learning techniques, thereby forming a robust hybrid control architecture. By thoroughly analyzing vehicle geometry and kinematics, an adaptive look-ahead mechanism is proposed to enhance safety and stability in driving scenarios, particularly under varying speed and curvature conditions. The integration of a deep reinforcement learning module facilitates the continuous optimization of the control strategy in dynamic environments, thereby improving the precision and efficiency of path tracking through real-time feedback adjustments. Additionally, initial policy iteration optimization and a linear decay of learning

Authorized licensed use limited to: American University of Beirut. Downloaded on January 28,2026 at 19:35:21 UTC from IEEE Xplore. Restrictions apply.

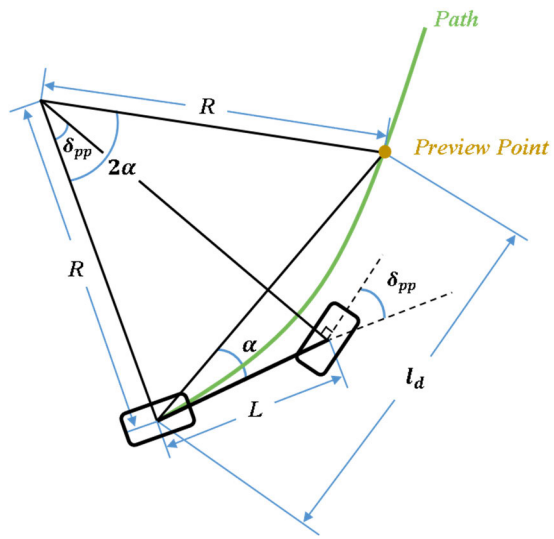


Fig. 2. Schematics on vehicle geometrics and kinematics.

rates have been implemented, which significantly enhance the convergence speed and stability of the training process. The subsequent sections will provide a detailed introduction of each component:

### A. Pure Pursuit Algorithm

In the context of vehicle path tracking, we often use a simplified model known as the two-wheel kinematic bicycle model. This model reduces the complexity of a four-wheel vehicle by representing it with two wheels: one at the front and one at the rear, connected by a wheelbase. This simplification captures the essential dynamics of the vehicle, making it easier to analyze and implement control algorithms like the Pure Pursuit algorithm.

The schematics of the vehicle geometry and kinematics, based on this two-wheel kinematic bicycle model, are illustrated in Fig. 2. Here, $R$ represents the turning radius, $\alpha$ is the angle between the line connecting the preview point and the rear axle center and the longitudinal axis of the vehicle, $L$ is the wheelbase, $\delta_{pp}$ is the front-wheel steering angle, and $l_d$ is the preview distance. By applying the law of sines, the following equation can be derived:

$$\frac{l_d}{\sin(2\alpha)} = \frac{R}{\sin(\pi/2 - \alpha)} \tag{1}$$

It can be derived from Fig. 2:

$$\tan(\delta_{pp}) = \frac{L}{R} \tag{2}$$

Then, the following can be obtained:

$$\delta_{pp} = \tan^{-1}\left(\frac{2L\sin(\alpha)}{l_d}\right) \tag{3}$$

In the Pure Pursuit algorithm, a preview point is identified on the tracking path at a distance $l_d$ from the current position of the vehicle. This preview point is used to calculate the necessary steering angle $\delta_{pp}$ to minimize the deviation from the desired path, ensuring accurate path tracking.

### B. Adaptive Look-Ahead Mechanism

The look-ahead distance is a crucial parameter in vehicle tracking control. It significantly affects the vehicle's safety and stability while driving. The design of the look-ahead distance needs to consider two main factors: the vehicle's speed and the road curvature.

First, the vehicle's speed is directly proportional to the look-ahead distance. At higher speeds, the driver or the autonomous driving system requires more time to react and adjust the direction. Therefore, to ensure safety and stability, the look-ahead distance should increase with speed. The look-ahead distance can be expressed as a function of speed:

$$l'_d = l_0 + k \cdot v \tag{4}$$

where $l_0$ represents the initial look-ahead distance (set to 6 meters in this study); $k$ is the coefficient related to speed; and $v$ is the vehicle's longitudinal speed in meters per second.

Second, in sharp turns, the look-ahead distance also needs to account for the road curvature. A larger curvature indicates a sharper turn, requiring the vehicle to adjust its path more quickly. Consequently, a shorter look-ahead distance is needed to improve the response time and path tracking accuracy. To make the curvature factor more effective in the calculation, we increase the curvature values by a factor of 10, as its values are relatively small compared to the speed. We incorporate the curvature factor into the look-ahead distance calculation formula:

$$l_d = l_0 + k \cdot v - k_d \cdot |c'| \tag{5}$$

where $k_d$ is the coefficient related to curvature; $c'$ is the curvature of the look-ahead point (at a distance of $l'_d$).

The coefficients $k$ and $k_d$ are dynamically output by the deep reinforcement learning module. These coefficients are adjusted according to different driving scenarios to optimize the look-ahead distance, thereby improving the vehicle's path tracking performance and driving safety.

### C. Deep Reinforcement Learning Module

Vehicle path tracking control can be viewed as a Markov decision process (MDP), and RL is an essential tool for solving MDP problems. The problem is formalized as a tuple $(S, A, P, R, \gamma)$. Here, $S$ is a finite set of states, $A$ is a finite set of actions, $P$ represents the state transition probabilities, $R$ is the reward function, and $\gamma \epsilon [0, 1]$ is the discount factor. The learner and decision-maker are referred to as the agent. Everything the agent interacts with, other than itself, is known as the environment. The agent and the environment interact at each step of a series of discrete time steps. At each time point $t$, the agent receives an environment state $S_t \in nS$ and chooses an action $A_t \in A$. After one-time step, the agent receives a numerical reward $R_{t+1} \in R$, and then transitions to a new state $S_{t+1}$. The task is defined as a finite horizon problem, with the horizon length being $T$ time steps. The agent's goal is to learn the optimal control policy $\pi(A_t|S_t;\theta_\pi)$, ultimately maximizing the expected return.

$$\pi^* = \arg\max_{\theta_\pi} E\left[\sum_{t=0}^{T-1} \gamma^t R_{t+1}\right] \tag{6}$$

Authorized licensed use limited to: American University of Beirut. Downloaded on January 28,2026 at 19:35:21 UTC from IEEE Xplore. Restrictions apply.

where the discount factor $\gamma$ is set to 0.99, which is commonly used in reinforcement learning to ensure that both immediate and future rewards are considered with a near-equal weight. A value close to 1 (such as 0.99) is chosen to allow the model to emphasize long-term rewards while still retaining the influence of short-term outcomes. This is a standard choice that balances the trade-off between immediate and delayed rewards, promoting stability in learning. $E(X)$ represents the expectation of the random variable $X$.

Several classic RL algorithms, such as PPO [25], DDPG [26], and deep Q-network [27], are identified. The PPO algorithm, introduced by OpenAI in 2017, is considered a state-of-the-art method in the field of RL and is a widely applicable algorithm. Therefore, in this study, the PPO algorithm was utilized to address the path tracking problem. The Advantage Function is a critical concept in RL, serving as a measure of the relative merit of a particular state and action compared with the average level. Its equation is expressed as:

$$A(s,a) = Q(s,a) - V(s) \tag{7}$$

where $Q(s,a)$ is the action-value, representing the value of taking action $a$ in state $s$, and $V(s)$ is the state-value function, representing the average value of being in state $s$.

Furthermore, to ensure that policy updates remain within a reasonable range, thereby guaranteeing the stability of the training, PPO employs the concept of a clipped surrogate objective as in Eq. (8).

$$L_t^{clip}(\theta) = \min(r_t(\theta)A_t, clip(r_t(\theta), 1-\varepsilon, 1+\varepsilon)A_t) \tag{8}$$

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} \tag{9}$$

where $r_t(\theta)$ represents the ratio of the probability of taking action $a_t$ in state $s_t$ under the current policy to the probability of taking the same action in the same state under the old policy. $\varepsilon$ is a hyperparameter used to control the clipping extent; $clip(r_t(\theta), 1-\varepsilon, 1+\varepsilon)$ is a clipping function that restricts $r_t(\theta)$ to the interval $[1-\varepsilon, 1+\varepsilon]$.

*1) State Space:* Initially, four key state variables are considered as in Fig. 3. These variables encompass the angle $\boldsymbol{\alpha}$ between the line connecting the preview point and the rear axle center $M$ and the longitudinal axis of the vehicle, the lateral deviation $d$ between the actual driving path and the tracking path, the yaw rate $\gamma$ (directly obtained from Simulink), and the vehicle's longitudinal velocity $v$. The consideration of $\alpha$ allows the agent to detect changes in the path ahead in advance, thereby making rational decisions. The variable $d$ informs the agent of the degree of deviation from the target path. The variable $\varphi$, which represents the yaw angle of the vehicle, measures the vehicle's heading and helps to understand its current orientation relative to the path; The variable $v$ informs the agent that the output steering angle is related to speed. The comprehensive consideration of these state variables helps in fully understanding and describing the dynamic behavior of the vehicle. Summarizing the abovementioned variables, the state space is expressed as:

$$S_1 = [\alpha, d, \varphi, v] \tag{10}$$

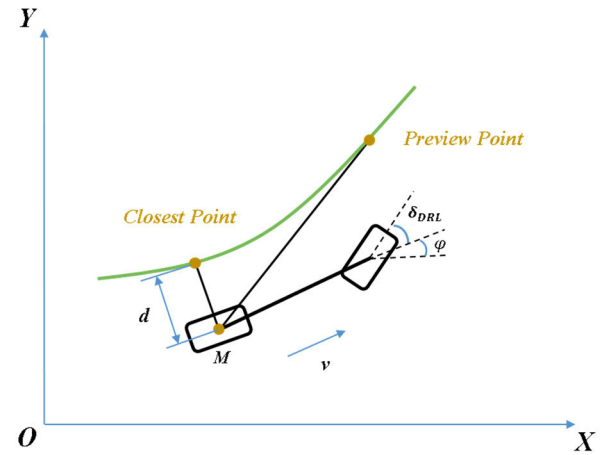


Fig. 3. Geometric representation of state variable.

Moreover, understanding the curvature changes between the current position and the path ahead is crucial for the agent to make sound decisions. Therefore, state $S_2$, which consists of the curvature $c$ at the vehicle's current tracking path point and the curvature $c'$ at the farthest point within the preview distance, was added to state $S_1$. This helps the agent better assess the curvature differences between the current and expected paths, thereby making more reasonable actions:

$$S_2 = [c, c'] \tag{11}$$

where $c$ and $c'$ represent the curvature at the current point and preview point, respectively.

By integrating Eqs. (10) and (11), the final state space $S$ can be obtained:

$$S = [S_1, S_2] = [\alpha, d, \varphi, v, c, c'] \tag{12}$$

*2) Action Space:*

$$A = [\delta_{DRL}, k, k_d] \tag{13}$$

where $\delta_{DRL}$ represents the front-wheel steering angle output by the DRL model, which is constrained to $\delta_{DRL} \epsilon [-20°, 20°]$ to ensure that the steering angle remains within the vehicle's operational limits and to prevent unnatural control behavior. Here, $k$ is the coefficient related to speed, $k_d$ is the coefficient related to curvature from Eq. (5).

*3) Reward Function:* The task of vehicle lateral control is to minimize the lateral displacement and angular error relative to the reference path [28], [29]. To achieve this goal, we constructed a reward function, as in Eq. (14), which considers angle $\alpha$ between the line connecting the preview point and the rear axle center and the longitudinal axis of the vehicle, as well as the lateral deviation $d$ between the actual driving path and the tracking path. Furthermore, to provide a continuous learning signal, we provide rewards at every step rather than at the end of an episode [30].

$$R = \cos(\alpha) - \lambda \sin(|\alpha|) - \frac{d}{d_{max}} \tag{14}$$

where $\alpha$ is the angle between the line connecting the preview point and the rear axle center and the longitudinal axis of the vehicle; $d$ is the lateral deviation between the actual

Authorized licensed use limited to: American University of Beirut. Downloaded on January 28,2026 at 19:35:21 UTC from IEEE Xplore. Restrictions apply.

driving path and the tracking path, and $d_{max}$ is the maximum allowable lateral deviation, which is set to 2.5 m. This value was chosen based on practical considerations, including typical road width, safety margins for vehicle control, and training efficiency. If this value is exceeded, then a new episode begins. Furthermore, $\lambda$ is a hyperparameter set to 1.

*4) Network Structure:* The PPO algorithm enhances the stability and efficiency of reinforcement learning by addressing policy optimization challenges through a well-structured network design comprising two primary components: Actor Network and Critic Network.

The Actor Network receives state representations as input and generates actions as output. For continuous actions, it utilizes a multivariate normal distribution. The network architecture comprises an input layer (State Dim), two hidden layers (64 units each with Tanh activation), and an output layer (Action Dim) with Tanh activation.

The Critic Network evaluates the value function, which mirrors the structure of the Actor Network. It consists of an input layer (State Dim), two hidden layers (64 units each with Tanh activation), and an output layer (Value) with a linear activation function.

The key features of PPO include the Clipping Mechanism and Advantage Estimation. The Clipping Mechanism, positioned between the Actor's hidden layers, constrains policy updates to mitigate destabilization and promote smoother convergence. Advantage Estimation within the Critic's hidden layers computes the advantage function, quantifying the benefit of actions under the current policy compared to a baseline, thereby facilitating more informed policy updates.

This network design, enhanced by the clipping mechanism and advantage estimation, underpins PPO's robustness and effectiveness in reinforcement learning. By balancing policy updates and leveraging advantage information, PPO facilitates stable and efficient policy optimization across diverse environments.

In Fig. 4, the Actor Network (in blue) and the Critic Network (in red) show the state dimension, hidden layers, and output layer. Connections between these layers illustrate the flow of information and activation functions. The Clipping Mechanism and Advantage Estimation are strategically placed within their respective networks to highlight their roles. Fig. 4 depicts the network structure of the PPO algorithm.

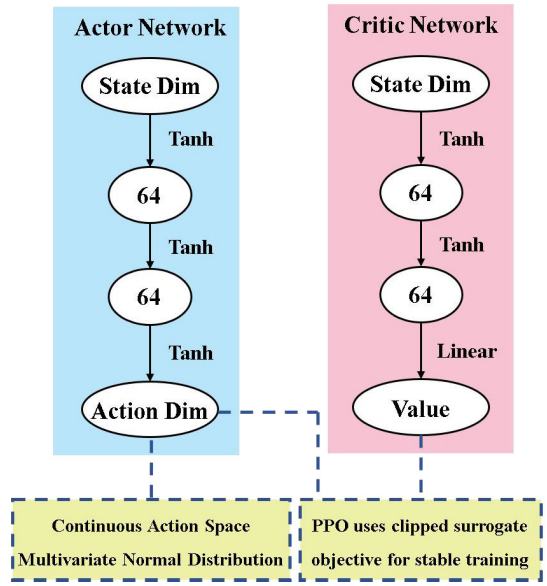


Fig. 4. Network structure.

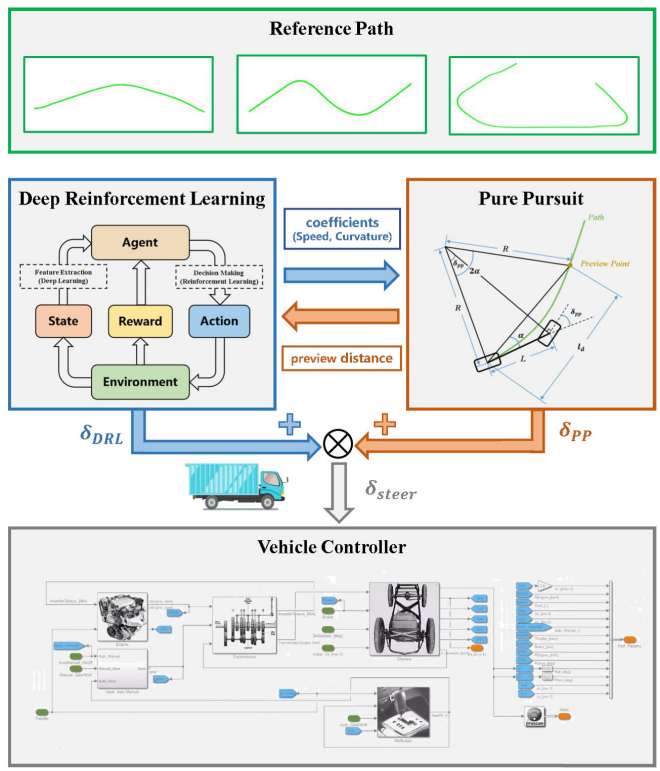


Fig. 5. Schematic diagram of the overall controller architecture.

## D. Hybrid Controller Architecture

The schematic diagram of the overall controller architecture is presented in Fig. 5. This architecture employs an innovative control strategy that integrates PP and PPO for enhanced path tracking. The PP algorithm functions as the primary control strategy, thereby enhancing interpretability and stability. It calculates the steering angle $\delta_{pp}$ necessary to adhere to the reference path using Eq. (3). The PPO algorithm operates concurrently with the PP algorithm to optimize control adjustments, generating a corrective steering angle $\delta_{DRL}$ to compensate for PP inaccuracies. The final steering angle $\delta_{steer}$ is executed by the vehicle controller.

$$\delta_{steer} = \delta_{PP} + \delta_{DRL} \tag{15}$$

Furthermore, the preview distance, a critical parameter for vehicle tracking control, is adaptively adjusted in accordance with the vehicle's speed and road curvature. Specifically, the DRL module supplies dynamic speed and curvature coefficients to the PP module, which subsequently calculates the adaptive preview distance using Eq. (5).

This architecture effectively combines both traditional and advanced control strategies, providing a robust and clear solution for path tracking in autonomous vehicles.

Authorized licensed use limited to: American University of Beirut. Downloaded on January 28,2026 at 19:35:21 UTC from IEEE Xplore. Restrictions apply.

TABLE I
CONVERGENCE METRICS FOR PPO ALGORITHM VARIANTS

| Algorithm | Convergence episode | Convergence reward |
|---|---|---|
| PPO_S1 | 475 | 0.72 |
| PPO_S1S2 | 567 | 0.77 |
| PPO_S1S2_IPIO | 643 | 0.82 |
| PPO_S1S2_LRLDM | 552 | 0.82 |
| PPO_S1S2_IPIO_LRLDM | 356 | 0.82 |

### E. Fusion Training Method

In RL, the initial policy is crucial for the performance and convergence speed of the algorithm. A well-designed initial policy significantly enhances the algorithm to quickly find better strategies, making the selection of an appropriate initial policy essential for the successful application of the algorithm. Therefore, a training technique called "Initial Policy Iterative Optimization" (IPIO) was introduced. In each completed tracking task episode, the average reward of that episode is compared to the previous one. By saving and loading neural network parameters from high-reward episodes, this technique ensures that each training round starts with a good initial policy. This approach accelerates the convergence speed of the algorithm.

Generally, the learning rate is initially set high to converge more quickly to a local optimum in the early stages of training. As training progresses, the learning rate gradually decreases, aiding the model to fine-tune parameters more precisely and avoid skipping over the global optimum. Consequently, the learning rate linear decay method (LRLDM) can improve training efficiency and accelerate model convergence. The decay process of the learning rate $l_r$ is implemented through a linear function:

$$l_r = \begin{cases} l_{r_0} - l_d, & if\ (t\%t_u == 0 \& l_r > l_{\min}) \\ l_{\min} & if\ (l_r \leq l_{\min}), \end{cases} \tag{16}$$

where $l_{r_0}$ represents the initial value of the learning rate, $l_{min}$ represents the minimum value of the learning rate, $t$ represents the total number of time steps, and $t_u$ represents the current update step.

*Validation of the Innovative Fusion Training Method:*
PPO_S1: The state space is defined as in Eq. (10), excluding curvature-related terms and without employing the IPIO and LRLDM methods.
PPO_S1S2: The state space is defined as in Eq. (12), and it does not use the IPIO and LRLDM methods.
PPO_S1S2_IPIO: The state space is defined in Eq. (12), but it does not use the LRLDM method.
PPO_S1S2_LRLDM: The state space is defined in Eq. (12), but it does not use the IPIO method.

Fig. 6 illustrates the training outcomes of five algorithms on an identical path, with the vehicle maintaining a constant speed. The results have been smoothed for clarity. The green line illustrates the outcome of our training, which integrating IPIO with the LRLDM. Compared to the separate utilization of these two methods, the combined training exhibits superior performance in convergence rounds, with negligible change in convergence rewards. Furthermore, compared to PPO_S1S2 without combined training, PPO_S1S2_IPIO_LRLDM, which utilized combined training, showed improvements in both convergence rounds and rewards, validating its effectiveness. Additionally, compared to the PPO_S1 algorithm, PPO_S1S2 also showed enhancements in convergence rounds and rewards, indicating that the added curvature information effectively enhances algorithm performance.

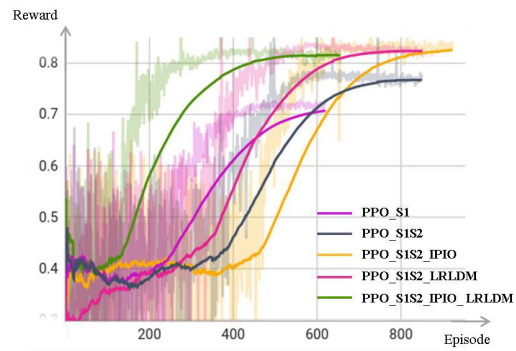


Fig. 6. Training performance of PPO algorithm variants.

TABLE II
PARAMETRIC SETTINGS OF DRL

| Parameter | Value |
|---|---|
| Sample time [s] | 0.05 |
| Max training steps | 2000000 |
| Max training steps per episode | 2000 |
| Initial variance | 0.6 |
| Decay rate of variance | 0.05 |
| Final fixed variance | 0.1 |
| The interval of variance decay steps | 8000 |
| Number of policy updates | 80 |
| Clip parameter | 0.2 |
| Discount factor | 0.99 |
| Init learning rate for actor network | 0.0027 |
| Decay rate of learning rate for actor network | 0.00015 |
| Minimum learning rate for actor network | 0.0003 |
| Init learning rate for critic network | 0.009 |
| Decay rate of learning rate for critic network | 0.0005 |
| Minimum learning rate for critic network | 0.001 |

### F. Experimental Hyperparameters

See Table II.

## IV. SIMULATION EXPERIMENTS

In this section, we thoroughly evaluate the proposed methods, highlighting the integration of theoretical constructs with practical applications through rigorous testing. Initially, we explored how our algorithms operate within a Software-in-the-Loop training framework based on PreScan. This framework provides a realistic environment that allows to refine and validate control strategies effectively. Then, an ablation study was conduct to systematically assess the impact of each algorithm component, clarifying their contributions to the overall performance enhancements. Comparative analysis was conducted against established control strategies, using a suite

Authorized licensed use limited to: American University of Beirut. Downloaded on January 28,2026 at 19:35:21 UTC from IEEE Xplore. Restrictions apply.

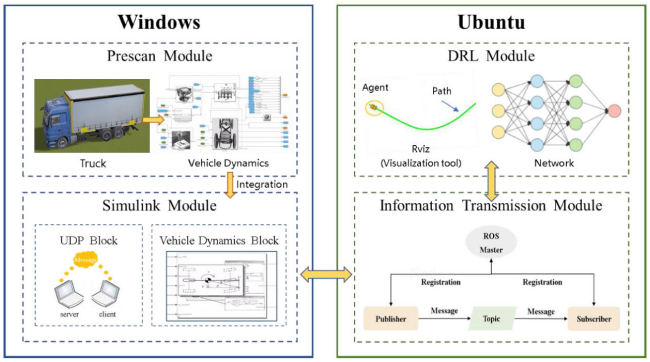


Fig. 7. Training framework and communication schematic.

of predefined metrics to assess the efficiency and robustness of the proposed methods under diverse conditions. All these not only demonstrate the capabilities of the proposed methods but also confirm their practical applicability in real-world scenarios.

### A. Software-in-the-Loop Training Framework Based on PreScan

The overall architecture of the training framework is illustrated in Fig. 7, which is divided into two main systems: Windows and Ubuntu. Each system internally comprises two key modules, namely, the PreScan module, Simulink module, information transmission module, and RL module. These modules perform distinct functions:

- **PreScan module**: This module is responsible for constructing the physical model of the truck. The interface for setting model parameters (Fig. 8) includes various parameters such as the test vehicle's wheelbase mass, front and rear axle distance, and center of mass location. This detailed model accurately describes the vehicle's behavior and dynamic characteristics, thereby improving the simulation accuracy of the path tracking system.
- **Simulink module**: This module sends vehicle position, speed, orientation angle, etc., to the information transmission layer and receives steering angle, brake, gear, and restart flag information.
- **Reinforcement learning module**: This module contains a visualization layer built with the visualization tool Rviz and an algorithm layer. Here, Rviz is used to visualize the training process of the agent, and the algorithm layer implements the RL algorithm, which is used for training the path tracking strategy of the autonomous vehicle.
- **Information transmission module**: As the intermediary for information transfer between the Simulink module and the RL module, it integrates ROS publish/subscribe nodes internally. It uses the UDP communication protocol for data transfer with the Simulink module and topic communication for data transfer with the RL layer. This module is responsible for acquiring vehicle state information from the Simulink module, which is used to compute the state space information for the RL module, and receiving steering angle, brake, gear, and restart flag information from the RL module to pass back to the Simulink module.

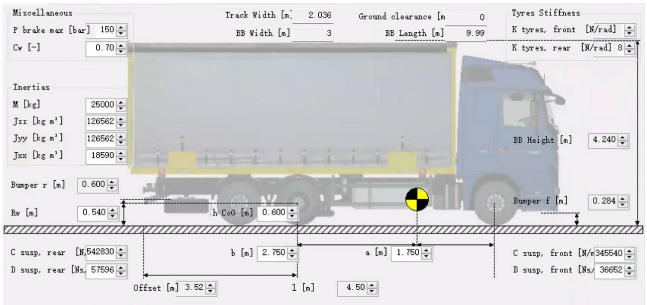


Fig. 8. Interface for setting vehicle model parameters.

Overall, this training framework integrates multiple functional modules, aiming to achieve high-fidelity simulation for autonomous vehicle path tracking algorithms.

### B. Comparative Algorithms and Evaluation Metrics

In comparative evaluation, a total of twelve benchmark control algorithms that embody different control methods and strategies were selected for analysis. These algorithms are introduced as follows:

- **LQR**: An optimal control method for linear systems, balancing precision and control energy by minimizing a quadratic performance index.
- **Fuzzy-PID**: Combines fuzzy logic with PID control for simple systems, enhancing performance and ease of adjustment for basic control tasks.
- **SMC**: A robust control method handling uncertainties and nonlinearities by forcing system states to follow a sliding surface.
- **MPC** [31]: This algorithm is applicable to complex, multivariable systems with strong robustness and performance, making it the preferred algorithm when precise control of the system is required.
- **Stanley** [32]: This algorithm is suitable for scenarios requiring high-precision path tracking, such as autonomous driving vehicles, capable of handling complex environments and paths, and it is a core algorithm for achieving autonomous driving.
- **DDPG**: A reinforcement learning algorithm for continuous action spaces, combining deep learning and policy gradients for high-dimensional control tasks.
- **PP6**: PP algorithm [33] with fixed preview distance, the preview distance is fixed at 6 meters, and it is designed to verify the effectiveness of varying speed.
- **PPV**: PP algorithm with variable preview distance varies with speed.
- **PPVC**: PP algorithm with variable preview distance varies with speed and curvature.
- **Pure_PPO**: Another naming format for the PPO_S1S2_IPIO_LRLDM method as described in Fig. 6.
- **PPV_Pure_PPO**: The combination of PPV and Pure_PPO algorithm.
- **PPVC_Pure_PPO**: The combination of PPVC and Pure_PPO algorithm.

Furthermore, to evaluate the performance of the selected algorithms, we selected three main evaluation metrics:

Authorized licensed use limited to: American University of Beirut. Downloaded on January 28,2026 at 19:35:21 UTC from IEEE Xplore. Restrictions apply.

Fig. 9. Paths with varied curvatures for performance evaluation.

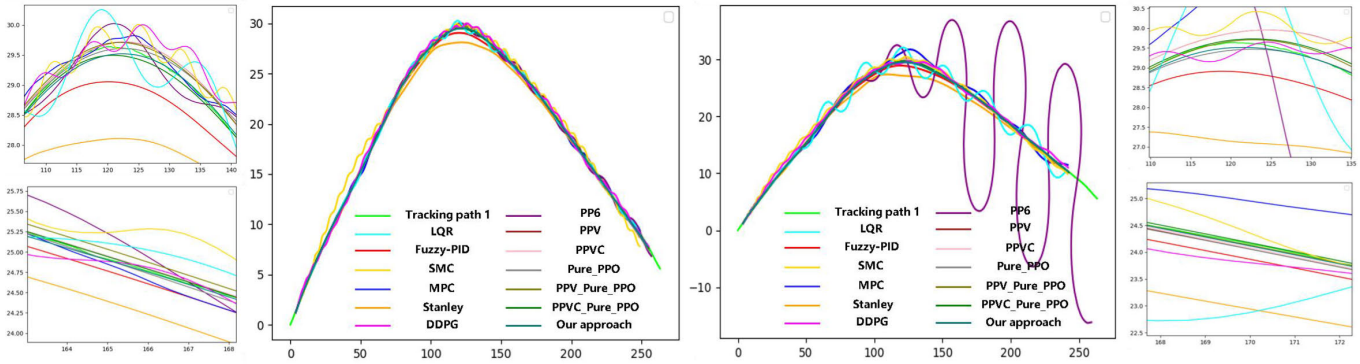


Fig. 10. Visualization of tracking performance for Path 1 and local magnified view.

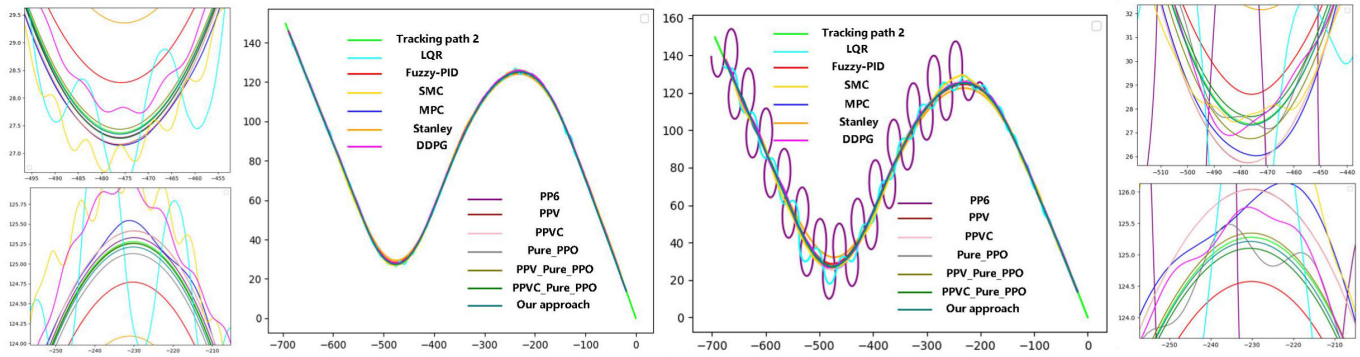


Fig. 11. Visualization of tracking performance for Path 2 and local magnified view.

- **Average lateral deviation:** This metric refers to the average deviation of the actual path from the desired path in the lateral direction, which aims to assess the average precision level of the control system in tracking the target path.
- **Maximum lateral deviation:** This metric refers to the maximum deviation of the actual path from the desired path in the lateral direction. It is used to assess the tracking performance and stability of the control system in extreme conditions. Since deviations can be both positive and negative, we compare the absolute values to determine the maximum lateral deviation.
- **Stability measure:** This metric aims to evaluate the stability of the control system in path tracking tasks. We use the variance of lateral deviation as a measure of stability. A smaller variance indicates a more stable tracking effect.

### *C. Experimental Results*

To comprehensively evaluate the tracking performance of our method and demonstrate its capability in handling scenarios with varying curvature (Fig. 9), we selected three distinct test paths: Parabolic Path 1, Sine Curve Path 2, and Sharp Triangular Path 3. Each path presents unique challenges: the parabolic path features continuously varying curvature, the sine curve path exhibits periodic curvature changes, and the sharp triangular path tests the method's robustness against abrupt curvature variations.

For thorough testing, we set the vehicle's longitudinal speed to 30 km/h and 60 km/h in Simulink, with the PID module employed to control the longitudinal speed accurately. The verification results are depicted in Figs. 10, 11, and 12, while the corresponding data for each path is detailed in Tables III and IV.

**Ablation Study Analysis**

To validate the effectiveness of our proposed hybrid control framework, we designed a series of ablation experiments comparing the performance of different algorithms and their variants. Specifically, we compared the following algorithms: PP6, PPV, PPVC, Pure_PPO, PPV_Pure_PPO, and PPVC_Pure_PPO, clearly demonstrating the advantages of combining PPO and PP.

*1) Effectiveness of Considering Speed:* Comparing the two, PPV outperforms PP6, especially at 60 km/h, with lower lateral deviation and higher stability. This indicates that considering the impact of vehicle speed on the look-ahead

Authorized licensed use limited to: American University of Beirut. Downloaded on January 28,2026 at 19:35:21 UTC from IEEE Xplore. Restrictions apply.

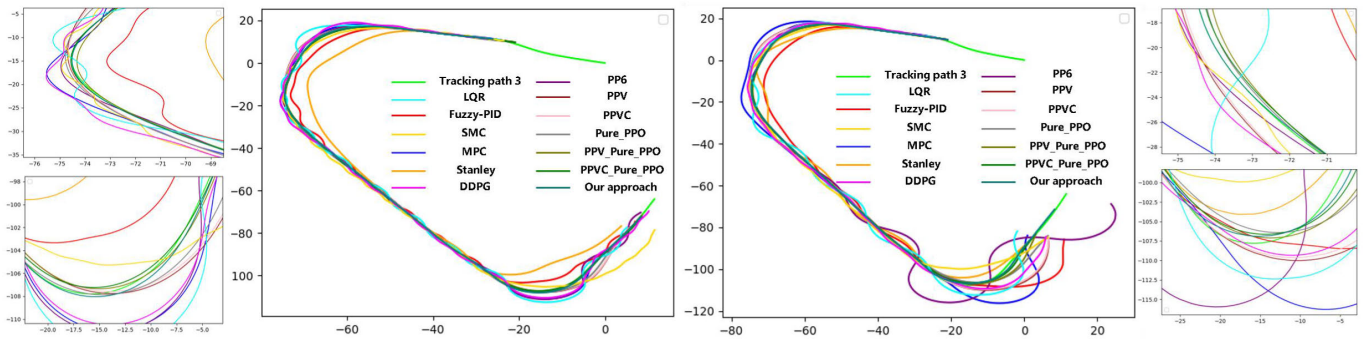


Fig. 12. Visualization of tracking performance for Path 3 and local magnified view.

TABLE III
COMPARATIVE ANALYSIS OF TRACKING PERFORMANCE (30KM/H)

| Algorithm | Average lateral deviation [m] | Maximum lateral deviation [m] | Stability measure |
|---|---|---|---|
| LQR | 0.180/0.537/1.075 | 0.642/2.050/5.190 | 0.052/0.435/2.380 |
| Fuzzy-PID | 0.163/0.234/1.031 | 0.590/0.915/4.991 | 0.036/0.123/1.501 |
| SMC | 0.525/0.649/1.304 | 1.991/1.852/6.895 | 0.357/0.588/4.121 |
| MPC | 0.154/0.114/0.576 | 0.502/0.478/3.732 | 0.035/0.021/0.807 |
| Stanley | 0.428/0.566/2.645 | 1.536/2.045/10.263 | 0.183/0.672/7.900 |
| DDPG | 0.251/0.473/0.785 | 0.639/0.959/3.200 | 0.069/0.048/0.709 |
| PP6 | 0.189/0.024/0.565 | 0.547/0.092/3.719 | 0.060/0.001/0.974 |
| PPV | 0.062/0.062/0.505 | 0.222/0.233/2.793 | 0.005/0.008/0.519 |
| PPVC | 0.061/0.060/0.470 | 0.222/0.220/2.626 | 0.005/0.008/0.453 |
| Pure_PPO | 0.114/0.106/0.470 | 0.299/0.160/1.905 | 0.018/0.002/0.213 |
| PPV_Pure_PPO | 0.070/**0.020**/0.131 | 0.249/0.099/0.711 | 0.004/0.001/0.041 |
| PPVC_Pure_PPO | 0.064/0.050/0.177 | 0.254/0.105/0.587 | 0.006/0.001/0.045 |
| Our approach | **0.047**/0.050/**0.083** | **0.160/0.087/0.384** | **0.003/0.000/0.012** |

Note: Each cell within the table includes the tracking results of the algorithm across three paths (1,2,3), with different results separated by a slash "/" symbol. Each result is rounded to three decimal places.

TABLE IV
COMPARATIVE ANALYSIS OF TRACKING PERFORMANCE (60KM/H)

| Algorithm | Average lateral deviation [m] | Maximum lateral deviation [m] | Stability measure |
|---|---|---|---|
| LQR | 0.968/2.357/1.342 | 2.792/11.778/6.513 | 1.697/10.459/3.803 |
| Fuzzy-PID | 0.196/0.300/2.428 | 0.770/1.329/13.999 | 0.057/0.222/16.734 |
| SMC | 0.662/1.226/1.087 | 1.776/4.342/7.951 | 0.364/2.415/2.958 |
| MPC | 0.525/0.477/2.152 | 2.300/1.589/12.751 | 0.419/0.392/9.460 |
| Stanley | 0.669/1.295/1.277 | 2.512/5.217/4.129 | 0.499/3.779/2.309 |
| DDPG | 0.400/0.446/1.075 | 1.092/1.701/5.581 | 0.144/0.265/1.898 |
| PP6 | 5.936/9.257/2.431 | 23.050/22.931/16.972 | 83.354/138.322/19.733 |
| PPV | 0.142/0.363/1.242 | 0.691/1.713/8.806 | 0.036/0.334/4.451 |
| PPVC | 0.138/0.362/1.158 | 0.686/1.688/8.355 | 0.034/0.331/3.937 |
| Pure_PPO | 0.066/0.218/0.516 | 0.214/0.910/3.541 | 0.005/0.083/0.999 |
| PPV_Pure_PPO | 0.053/0.129/0.362 | 0.218/0.645/2.832 | 0.005/0.033/0.445 |
| PPVC_Pure_PPO | 0.090/0.134/0.226 | 0.314/0.329/1.500 | 0.013/0.022/0.149 |
| Our approach | **0.042/0.045/0.170** | **0.136/0.175/0.952** | **0.003/0.002/0.069** |

distance improves path tracking accuracy, particularly at high speeds.

*2) Effectiveness of Considering Curvature:* PPVC shows higher accuracy compared to PPV, especially on paths with large curvatures. Specifically, in sharp turns, considering the curvature effectively reduces lateral deviation.

*3) Benefits of Combining PPO With PP:* PPV_Pure_PPO outperforms both Pure_PPO and PPV in handling complex environments, such as high-speed turns. Specifically, PPO helps the algorithm make precise control decisions at high speeds, while PPV provides a stable look-ahead mechanism, significantly improving path tracking accuracy and stability.

*4) Effectiveness of the Adaptive Look-Ahead Mechanism:* Compared to PPVC_Pure_PPO, although both approaches consider speed and curvature variations, PPVC_Pure_PPO uses fixed coefficients for adjusting the look-ahead distance, which cannot dynamically adapt to real-time conditions. As a result, in complex or sharp turns, the fixed coefficients for look-ahead distance may not match the actual road conditions accurately, leading to larger lateral deviations and instability.

In contrast, in our algorithm, the look-ahead distance is dynamically adjusted based on the vehicle's real-time speed and curvature, significantly improving tracking accuracy, especially on sharp turns. For instance, at 60 km/h, the maximum lateral deviation of our approach is much smaller than that of PPVC_Pure_PPO, with a reduction of about 40% in maximum deviation on some test paths. Our stability metric (i.e., the variance of lateral deviation) is also significantly lower than that of PPVC_Pure_PPO, demonstrating the stronger stability of our method in complex environments.

**Comparison with Other Algorithms**

On various experimental paths (e.g., parabolic path, sine curve path, and sharp triangular path), the superior performance of our method is closely related to the dynamic adjustment of the look-ahead mechanism. Especially on sharp turns, fixed look-ahead distances cannot quickly respond to changes in road curvature, whereas our adaptive look-ahead

Authorized licensed use limited to: American University of Beirut. Downloaded on January 28,2026 at 19:35:21 UTC from IEEE Xplore. Restrictions apply.

mechanism adjusts the look-ahead point according to real-time conditions, improving tracking accuracy.

When compared with other algorithms (e.g., MPC and Stanley), although they also perform well on complex paths, they cannot achieve the same stability and accuracy under extreme speed and curvature conditions. Our method shows significant advantages, particularly on the parabolic and sharp triangular paths.

Main Issue with LQR: LQR struggles to handle complex, nonlinear systems (e.g., sharp turns) and does not adapt well to varying speeds and curvatures.

Limitations of Fuzzy-PID: Fuzzy-PID may be significantly affected by large disturbances when faced with high speeds and complex paths, leading to reduced accuracy and stability, especially in scenarios that require high control precision.

Drawbacks of SMC: Although SMC offers good robustness, its switching control mode may lead to discontinuous control effects at high speeds, making path tracking less smooth. This can impact stability and accuracy, particularly at high speeds.

Despite some deviations under extreme conditions, our method maintains a lower level of maximum deviation compared to other algorithms (e.g., LQR, DDPG), demonstrating its superior robustness.

It is also worth noting that our algorithm not only performs well in experimental results but also achieves good path tracking performance without the need for extensive manual tuning. Compared to traditional control methods (e.g., PID, MPC), our method reduces the reliance on parameter tuning through adaptive mechanisms and PPO dynamic adjustments, lowering the application threshold and improving the algorithm's practicality and scalability.

## V. Real Experiments

Furthermore, this study conducted real experiments using a proportional scaled intelligent vehicle chassis. This section provides an introduction to the test platform, including the autonomous driving system and the controller of the intelligent vehicle chassis. Additionally, it covers the design of the experimental scenarios and the evaluation of the test results.

### A. Introduction and Rationality Analysis of the Test Platform

The test platform encompasses the intelligent vehicle chassis and its controller. This section justifies the selection of the proportional scaled intelligent vehicle by elaborating on its scientific, economic, safety, and experimental feasibility aspects.

*1) Chassis Introduction:* The proportional scaled intelligent vehicle chassis used in this study is shown in Fig. 13. The chassis employs an Ackermann steering mechanism, designed to ensure that the wheels on the inside of a turn rotate at different angles to match the intended turning radius. This mechanism reduces tire slippage and wear during turns by aligning the wheels' turning angles with the vehicle's path, effectively simulating the steering mechanism of a real freight truck.

The vehicle is equipped with high-precision sensors and computational units, enabling it to perform tasks such as autonomous navigation, path planning, and path tracking. The control system, based on mature autonomous driving algorithms, processes sensor data in real-time to ensure stable operation in complex environments.

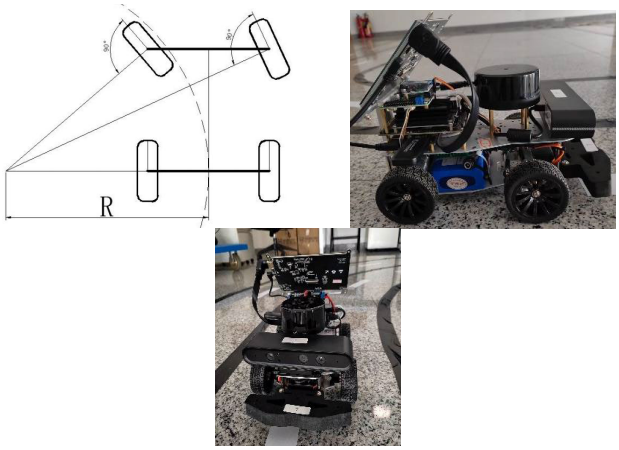


Fig. 13. Proportionally scaled intelligent vehicle with Ackermann steering mechanism.

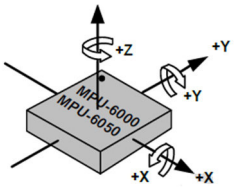


Fig. 14. Illustration of the MPU6050 accelerometer and gyroscope.

For positioning, the intelligent vehicle uses a sensor fusion approach that integrates data from an IMU sensor and odometry. The STM32 controller collects IMU data from the MPU6050 sensor, which includes a three-axis accelerometer and a three-axis gyroscope, as shown in Fig. 14. The sensor data is then processed by the robot_pose_ekf node within the Robot Operating System (ROS), which combines the IMU measurements with odometry information to produce a more accurate pose estimation, known as odom_combined. This approach enhances better precision for the vehicle's motion estimation, particularly in environments where GPS signals may be unreliable. While Lidar and ultrasonic sensors are commonly used for precise positioning and path tracking [34], we chose the IMU and odometry fusion method, as it facilitates real-time, high-frequency data processing and is well-suited for the scale of our experimental setup.

Although the IMU-based approach is not the traditional method involving Lidar or ultrasonic sensors, it is often complemented by additional sensor fusion techniques to achieve enhanced localization accuracy. In our case, the odom_combined data is further refined by integrating Lidar data for localization adjustments during navigation with the amcl or map_matching nodes. This hybrid approach aligns well with the vehicle's operational requirements, offering a robust solution for path tracking in controlled experimental settings.

*2) Controller Introduction:* The proportionally scaled intelligent vehicle used in the experiment is equipped with two core controllers: one ROS master controller and one STM32

Authorized licensed use limited to: American University of Beirut. Downloaded on January 28,2026 at 19:35:21 UTC from IEEE Xplore. Restrictions apply.

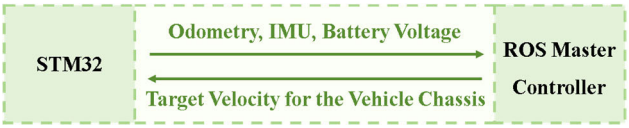


Fig. 15. Relationship between STM32 and ROS Master.

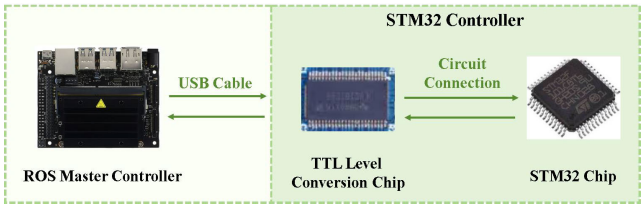


Fig. 16. Communication between ROS Master and STM32.

controller. The ROS system operates on a Jetson Nano, handling functions such as sensor data collection, path planning, and message communication. Meanwhile, the STM32 controller is responsible for tasks including the collection of odometry information (ODOM), gyroscope data (IMU), and motor control. Specifically, the odometry data consists of the rotational speed of each motor, while the gyroscope data includes the angular velocities and accelerations along the x, y, and z axes.

As depicted in Fig. 15, the relationship between these two controllers is such that the ROS master receives sensor data from the STM32 controller and then sends the target velocity for the vehicle chassis to the STM32 controller. The STM32 controller then uses this target velocity, along with the kinematic relationships of the vehicle chassis, to control the motors and servos, ensuring the vehicle travels at the desired velocity along each axis.

To achieve the aforementioned functionalities, communication between the two controllers is essential. As shown in Fig. 16, the communication connection involves the ROS master connecting to a TTL level conversion chip via a USB cable. This setup ensures consistent logic level conversion between the two controllers, which is then connected to the STM32 chip through a circuit.

*3) Rationale for Using Proportionally Scaled Intelligent Vehicles:*

*a) Scientific validity:* Testing path tracking algorithms with a proportionally scaled intelligent vehicle provides a high level of scientific validity. The Ackermann steering mechanism, commonly used in real freight trucks, can be accurately replicated on a smaller scale. This allows for realistic and reliable testing of path tracking algorithms, as the steering characteristics and dynamic behavior of the scaled vehicle closely mimic those of full-sized trucks.

*b) Economic efficiency:* Conducting experiments with full-sized freight trucks involves substantial costs, including manufacturing, maintenance, fuel, and venue expenses. In contrast, using a proportionally scaled intelligent vehicle is cost-effective, significantly reducing experimental expenses. This economic advantage enables more frequent experiments and tests, thereby enhancing the efficiency of algorithm development and validation.

*c) Safety:* Testing path tracking algorithms on actual freight trucks poses significant risks, especially when the algorithms are not yet fully developed. Using a scaled intelligent vehicle for testing allows experiments to be conducted in a controlled environment, greatly minimizing experimental risks and ensuring the safety of both personnel and equipment.

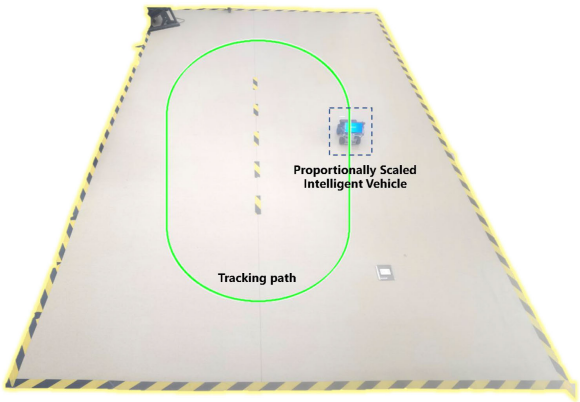


Fig. 17. Experimental scenario.

*d) Feasibility of experiments:* The compact size of the intelligent vehicle requires minimal experimental space, allowing tests to be conducted in laboratories or small test sites. This flexibility simplifies experiment design and implementation, enabling efficient algorithm testing within limited space and resource conditions. Additionally, the high-precision sensors and flexible control system of the intelligent vehicle facilitate a variety of complex experiments, fully meeting the testing needs of path tracking algorithms.

In summary, using a proportionally scaled intelligent vehicle for testing freight truck path tracking algorithms offers significant advantages in terms of scientific validity, economic efficiency, safety, and experimental feasibility. It provides a reliable and efficient experimental platform for algorithm development and validation.

## *B. Experimental Design and Test Results*

*1) Experimental Scene Design:* Based on the existing experimental conditions, we designed a real-world scenario that includes a rectangular area, as illustrated in the Fig. 17. The boundaries of the experimental area are constructed with stickers, with dimensions of 5m $\times$ 3.2m. Within this area, a 1.9m $\times$ 0.05m sticker is placed in the middle, serving as a barrier (acting as a "wall"). A symmetric trajectory was generated within this area, incorporating two straight segments and two curved segments, resembling a shape composed of two arcs and two straight lines centered around a middle horizontal line within the rectangle. This design allows for a comprehensive evaluation of the control algorithms' performance by including both straight paths and sharp turns while ensuring that the vehicle remains within the boundary and avoids collisions with the barrier.

Furthermore, to evaluate the performance of the selected algorithms, we selected three main evaluation metrics:

**1. Average lateral deviation:** This metric refers to the average deviation of the actual path from the desired path in the lateral direction, which aims to assess the average precision level of the control system in tracking the target path.

Authorized licensed use limited to: American University of Beirut. Downloaded on January 28,2026 at 19:35:21 UTC from IEEE Xplore. Restrictions apply.

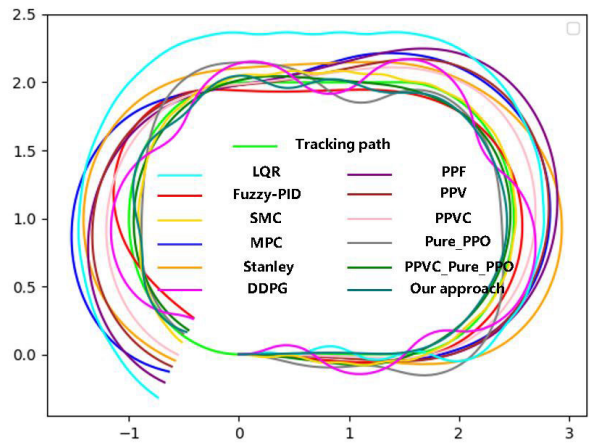


Fig. 18. Visualization of path tracking control performance (0.5m/s).

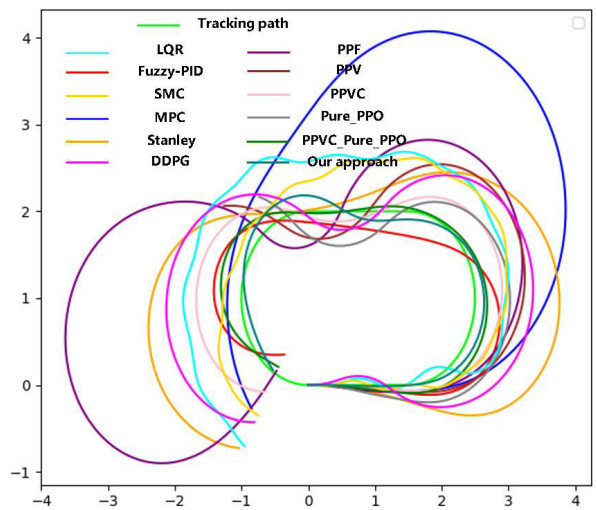


Fig. 19. Visualization of path tracking control performance (1m/s).

**2. Maximum lateral deviation:** This metric refers to the maximum deviation of the actual path from the desired path in the lateral direction. It is used to assess the tracking performance and stability of the control system in extreme conditions. Since deviations can be both positive and negative, we compare the absolute values to determine the maximum lateral deviation.

**3. Stability measure:** This metric aims to evaluate the stability of the control system in path tracking tasks. We use the variance of lateral deviation as a measure of stability. A smaller variance indicates a more stable tracking effect.

*2) Test Results:* This experiment primarily evaluates the real-world path tracking performance of the vehicle's reinforcement learning-based strategy. Due to site limitations and safety considerations, the vehicle speed was set to 0.5 m/s and 1 m/s. Various control algorithms were implemented on the real vehicle for comparative analysis. The results are presented in Fig. 18 and Fig. 19, with detailed metrics in TABLE V and TABLE VI.

LQR: The LQR algorithm, known for optimal control, performed reasonably well with an average lateral deviation of 0.532 m and a maximum of 0.953 m at 1 m/s. It maintained good stability with a measure of 0.097, outperforming more complex models like MPC.

TABLE V
COMPARATIVE ANALYSIS OF TRACKING PERFORMANCE (0.5M/S)

| Algorithm | Average lateral deviation [m] | Maximum lateral deviation [m] | Stability measure |
|---|---|---|---|
| LQR | 0.296 | 0.510 | 0.025 |
| Fuzzy-PID | 0.066 | 0.178 | 0.006 |
| SMC | 0.035 | 0.094 | 0.002 |
| MPC | 0.215 | 0.538 | 0.040 |
| Stanley | 0.216 | 0.441 | 0.032 |
| DDPG | 0.098 | 0.230 | 0.012 |
| PPF | 0.211 | 0.439 | 0.037 |
| PPV | 0.169 | 0.357 | 0.022 |
| PPVC | 0.101 | 0.220 | 0.009 |
| Pure_PPO | 0.087 | 0.192 | 0.010 |
| PPVC_Pure_PPO | 0.040 | **0.078** | **0.001** |
| Our approach | **0.039** | 0.095 | **0.001** |

Note: PPF represents the pure pursuit algorithm with a fixed preview distance.

TABLE VI
COMPARATIVE ANALYSIS OF TRACKING PERFORMANCE (1M/S)

| Algorithm | Average lateral deviation [m] | Maximum lateral deviation [m] | Stability measure |
|---|---|---|---|
| LQR | 0.532 | 0.953 | 0.097 |
| Fuzzy-PID | 0.195 | 0.427 | 0.049 |
| SMC | 0.342 | 0.620 | 0.046 |
| MPC | 0.981 | 2.127 | 0.594 |
| Stanley | 0.728 | 1.447 | 0.288 |
| DDPG | 0.557 | 1.136 | 0.171 |
| PPF | 1.008 | 2.688 | 0.947 |
| PPV | 0.369 | 0.809 | 0.138 |
| PPVC | 0.272 | 0.680 | 0.057 |
| Pure_PPO | 0.262 | 0.526 | 0.076 |
| PPVC_Pure_PPO | 0.107 | 0.327 | 0.012 |
| Our approach | **0.074** | **0.192** | **0.008** |

Fuzzy-PID: The Fuzzy-PID controller excelled in low-speed scenarios (0.5 m/s), achieving an average lateral deviation of 0.066 m and a maximum of 0.178 m. It demonstrated strong accuracy and stability with minimal tuning, outperforming MPC and Stanley in these aspects.

SMC: The SMC method showed robustness, especially in handling disturbances. At 0.5 m/s, it had an average deviation of 0.035 m and a maximum of 0.094 m, with a stability measure of 0.002, excelling in precision.

Authorized licensed use limited to: American University of Beirut. Downloaded on January 28,2026 at 19:35:21 UTC from IEEE Xplore. Restrictions apply.

MPC: The MPC algorithm is robust but didn't always outperform simpler models like PID, possibly due to tuning challenges or model inaccuracies. Its theoretical advantages were not fully realized in practical testing.

Stanley: The Stanley algorithm performed well in high-speed maneuvers but showed occasional minor deviations in sharp turns.

PPF: The PPF algorithm, with a fixed preview distance, struggled with rapidly changing speeds and curved paths, highlighting the need for adaptive preview distance based on speed and curvature.

PPV: The PPV algorithm adjusted the preview distance based on speed, showing improved performance over PPF in varying speed scenarios, resulting in more accurate tracking.

PPVC: The PPVC algorithm, adjusting for both speed and curvature, performed well in high-curvature turns and varying speeds, making it effective for complex paths.

PPVC_Pure_PPO: The PPVC_Pure_PPO algorithm adjusts preview distance based on speed and curvature, but its fixed adjustment limits performance in dynamic or sharp-turn scenarios. While it outperforms traditional methods, it is less accurate in complex environments compared to algorithms like Our Approach, which dynamically adjusts preview distance in real-time.

Our Approach: Our approach, with an adaptive look-ahead mechanism, dynamically adjusts the preview distance based on speed and curvature. It consistently outperformed other algorithms in terms of lateral deviation and stability at both 0.5 m/s and 1 m/s. This dynamic adaptation makes it especially effective in high-curvature and complex paths, where fixed-parameter methods (like PPVC_Pure_PPO) may not perform as well.

Based on the experimental results, our method consistently excelled across three key metrics: average lateral deviation, maximum lateral deviation, and stability measure. The algorithm's performance closely aligned with simulation results, confirming its reliability in both simulated and real-world settings. Furthermore, the algorithm demonstrated exceptional control accuracy in maintaining lateral precision, particularly in high-curvature turns, while also showcasing robustness in handling varying speeds and complex path conditions.

## VI. Conclusion and Future Work

This study presents a hybrid path tracking control architecture that integrates the interpretability of the Pure Pursuit (PP) method with the adaptability of the Proximal Policy Optimization (PPO) algorithm. Validation was conducted using a high-fidelity simulation framework based on PreScan, Simulink, and ROS, demonstrating significant improvements in tracking accuracy and robustness across diverse test paths. Real-world experiments with a proportionally scaled intelligent vehicle further affirmed the effectiveness of our approach, suggesting its potential adaptation for full-scale autonomous trucks pending additional testing on larger vehicles.

However, the proposed approach also has certain limitations. First, the effectiveness of the hybrid control strategy in real-world environments is subject to external disturbances such as sensor noise, unpredictable environmental changes, and vehicle wear, which were not fully accounted for in our current simulation and scaled vehicle experiments. Additionally, the transition from scaled experiments to full-scale trucks may introduce new challenges in handling dynamics and sensor integration, which require further investigation. Moreover, the sudden change in reference path curvature can affect the path tracking performance of autonomous trucks [35].

In future work, we plan to address these limitations by conducting more comprehensive real-world experiments using full-scale autonomous trucks, incorporating more advanced sensors and perception algorithms to improve environmental awareness and enhance decision-making capabilities. Additionally, testing the vehicle under more complex and extreme conditions (e.g., adverse weather, challenging terrain) will be essential to further refine and optimize the control strategy [36], [37]. This iterative process aims to bridge the gap between theoretical research and practical application, ultimately contributing to the deployment of more efficient and safer autonomous trucks in the logistics industry.

## References


[1] Y. Wiseman, "Autonomous vehicles," in *Encyclopedia of Information Science and Technology*, vol. 1, 5th ed. Hershey, PA, USA: IGI Global, 2020, ch. 1, pp. 1–11. [Online]. Available: https://u.cs.biu.ac.il/~wisemay/Autonomous-Vehicles-Encyclopedia.pdf

[2] A. Hasiri and A. Kermanshah, "Exploring the role of autonomous trucks in addressing challenges within the trucking industry: A comprehensive review," *Systems*, vol. 12, no. 9, p. 320, Aug. 2024.

[3] J. Han et al., "Uniform finite time safe path tracking control for obstacle avoidance of autonomous vehicle via barrier function approach," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 8, pp. 9512–9523, Aug. 2024, doi: 10.1109/TITS.2024.3384444.

[4] X. Li, X. Gong, Y.-H. Chen, J. Huang, and Z. Zhong, "Integrated path planning-control design for autonomous vehicles in intelligent transportation systems: A neural-activation approach," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 7, pp. 7602–7618, Jul. 2024, doi: 10.1109/TITS.2024.3353824.

[5] G. Wang, L. Liu, Y. Meng, Q. Gu, and G. Bai, "Integrated path tracking control of steering and braking based on holistic MPC," *IFAC-PapersOnLine*, vol. 54, no. 2, pp. 45–50, 2021, doi: 10.1016/j.ifacol.2021.06.007.

[6] G. Wang, L. Liu, Y. Meng, S. Wang, Q. Gu, and G. Bai, "MHE-based friction coefficient estimation and compensation method for path tracking control," in *Proc. IEEE 25th Int. Conf. Intell. Transp. Syst. (ITSC)*, Macau, China, Oct. 2022, pp. 146–152.

[7] D. Chu, H. Li, C. Zhao, and T. Zhou, "Trajectory tracking of autonomous vehicle based on model predictive control with PID feedback," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 2, pp. 2239–2250, Feb. 2023, doi: 10.1109/TITS.2022.3150365.

[8] J. Dong, S. Chen, Y. Li, R. Du, A. Steinfeld, and S. Labi, "Facilitating connected autonomous vehicle operations using space-weighted information fusion and deep reinforcement learning based control," 2020, *arXiv:2009.14665*.

[9] Y. Cao, K. Ni, T. Kawaguchi, and S. Hashimoto, "Path following for autonomous mobile robots with deep reinforcement learning," *Sensors*, vol. 24, no. 2, p. 561, Jan. 2024, doi: 10.3390/s24020561.

[10] Y. Tian, X. Cao, K. Huang, C. Fei, Z. Zheng, and X. Ji, "Learning to drive like human beings: A method based on deep reinforcement learning," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 7, pp. 6357–6367, Jul. 2022, doi: 10.1109/TITS.2021.3055899.

[11] R. Marino, S. Scalzi, and M. Netto, "Nested PID steering control for lane keeping in autonomous vehicles," *Control Eng. Pract.*, vol. 19, no. 12, pp. 1459–1467, Dec. 2011.

[12] B. Zhang, J. Cheng, Y. Zhang, C. Zhang, and G. Shen, "Path tracking of autonomous tractor based on model-free adaptive control," *Proc. Inst. Mech. Eng., D, J. Automobile Eng.*, vol. 238, no. 4, pp. 749–759, Mar. 2024.


Authorized licensed use limited to: American University of Beirut. Downloaded on January 28,2026 at 19:35:21 UTC from IEEE Xplore. Restrictions apply.

[13] Q. Shi and H. Zhang, "Road-curvature-range-dependent path following controller design for autonomous ground vehicles subject to stochastic delays," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 10, pp. 17440–17450, Oct. 2022, doi: 10.1109/TITS.2022.3157059.

[14] P. Stano et al., "Model predictive path tracking control for automated road vehicles: A review," *Annu. Rev. Control*, vol. 55, pp. 194–236, Jan. 2023, doi: 10.1016/j.arcontrol.2022.11.001.

[15] M. Rokonuzzaman, N. Mohajer, and S. Nahavandi, "Effective adoption of vehicle models for autonomous vehicle path tracking: A switched MPC approach," *Vehicle Syst. Dyn.*, vol. 61, no. 5, pp. 1236–1259, May 2023, doi: 10.1080/00423114.2022.2071300.

[16] S. Xu and H. Peng, "Design, analysis, and experiments of preview path tracking control for autonomous vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 21, no. 1, pp. 48–58, Jan. 2020, doi: 10.1109/TITS.2019.2892926.

[17] N. A. I. Ruslan, N. H. Amer, K. Hudha, Z. A. Kadir, S. A. F. M. Ishak, and S. M. F. S. Dardin, "Modelling and control strategies in path tracking control for autonomous tracked vehicles: A review of state of the art and challenges," *J. Terramechanics*, vol. 105, pp. 67–79, Feb. 2023, doi: 10.1016/j.jterra.2022.10.003.

[18] L. Yu, X. Yan, Z. Kuang, B. Chen, and Y. Zhao, "Driverless bus path tracking based on fuzzy pure pursuit control with a front axle reference," *Appl. Sci.*, vol. 10, no. 1, p. 230, Dec. 2019, doi: 10.3390/app10010230.

[19] J. Ahn, S. Shin, M. Kim, and J. Park, "Accurate path tracking by adjusting look-ahead point in pure pursuit method," *Int. J. Automot. Technol.*, vol. 22, no. 1, pp. 119–129, Feb. 2021, doi: 10.1007/s12239-021-0013-7.

[20] I.-M. Chen and C.-Y. Chan, "Deep reinforcement learning based path tracking controller for autonomous vehicle," *Proc. Inst. Mech. Eng., D, J. Automobile Eng.*, vol. 235, nos. 2–3, pp. 541–551, Feb. 2021, doi: 10.1177/0954407020954591.

[21] J. Moon, S. Papaioannou, C. Laoudias, P. Kolios, and S. Kim, "Deep reinforcement learning multi-UAV trajectory control for target tracking," *IEEE Internet Things J.*, vol. 8, no. 20, pp. 15441–15455, Oct. 2021, doi: 10.1109/JIOT.2021.3073973.

[22] E. Okafor, D. Udekwe, Y. Ibrahim, M. B. Mu'azu, and E. G. Okafor, "Heuristic and deep reinforcement learning-based PID control of trajectory tracking in a ball-and-plate system," *J. Inf. Telecommun.*, vol. 5, no. 2, pp. 179–196, Apr. 2021, doi: 10.1080/24751839.2020.1833137.

[23] S. Wang, X. Yin, P. Li, M. Zhang, and X. Wang, "Trajectory tracking control for mobile robots using reinforcement learning and PID," *Iranian J. Sci. Technol., Trans. Electr. Eng.*, vol. 44, no. 3, pp. 1059–1068, Sep. 2020, doi: 10.1007/s40998-019-00286-4.

[24] Y. Zhao, X. Qi, Y. Ma, Z. Li, R. Malekian, and M. A. Sotelo, "Path following optimization for an underactuated USV using smoothly-convergent deep reinforcement learning," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 10, pp. 6208–6220, Oct. 2021, doi: 10.1109/TITS.2020.2989352.

[25] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.

[26] T. P. Lillicrap et al., "Continuous control with deep reinforcement learning," 2015, *arXiv:1509.02971*.

[27] V. Mnih et al., "Playing Atari with deep reinforcement learning," 2013, *arXiv:1312.5602*.

[28] L. Claussmann, M. Revilloud, D. Gruyer, and S. Glaser, "A review of motion planning for highway autonomous driving," *IEEE Trans. Intell. Transp. Syst.*, vol. 21, no. 5, pp. 1826–1848, May 2020, doi: 10.1109/TITS.2019.2913998.

[29] L. Chen et al., "Milestones in autonomous driving and intelligent vehicles—Part I: Control, computing system design, communication, HD map, testing, and human behaviors," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 53, no. 9, pp. 5831–5847, Sep. 2023, doi: 10.1109/TSMC.2023.3276218.

[30] D. Li, D. Zhao, Q. Zhang, and Y. Chen, "Reinforcement learning and deep learning based lateral control for autonomous driving," 2018, *arXiv:1810.12778*.

[31] B. Kouvaritakis and M. Cannon, *Model Predictive Control*. Berlin, Germany: Springer, 2016.

[32] S. Thrun et al., "Stanley: The robot that won the DARPA grand challenge," *J. Field Robot.*, vol. 23, no. 9, pp. 661–692, 2006, doi: 10.1002/rob.20147.

[33] R. C. Conlter, "Implementation of the pure pursuit path tracking algorithm," Camegie Mellon Univ., Pittsburgh, PA, USA, Tech. Rep. CMU-RI-TR- 92-01, 1992.

[34] Y. Wiseman, "Ancillary ultrasonic rangefinder for autonomous vehicles," *Int. J. Secur. Appl.*, vol. 12, no. 5, pp. 49–58, Sep. 2018. [Online]. Available: https://u.cs.biu.ac.il/~wisemay/ijsia2018.pdf

[35] T. Liu, Y. Shen, and K. Wang, "Path tracking control for autonomous truck with dual modular chassis," *Automot. Innov.*, vol. 6, no. 4, pp. 558–570, Nov. 2023.

[36] Z. Xu, C. Wang, T. Jiang, Z. Liu, and N. Zheng, "Impediments to environmental awareness in autonomous driving systems and its effect on user adoption," *IEEE Trans. Intell. Vehicles*, early access, Mar. 8, 2024, doi: 10.1109/TIV.2024.3373773.

[37] W. Wang, Y. Zhang, C. Yang, T. Qie, and M. Ma, "Adaptive model predictive control-based path following control for four-wheel independent drive automated vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 9, pp. 14399–14412, Sep. 2022, doi: 10.1109/TITS.2021.3128268.

**Zhixuan Han** is currently pursuing the Ph.D. degree with the School of Transportation Science and Engineering, Beihang University, Beijing, China. His research interests include decision making, trajectory planning, and trajectory tracking control of intelligent vehicles.

**Peng Chen** (Member, IEEE) received the Ph.D. degree from Nagoya University, Japan, in 2012. He is currently an Associate Professor with the School of Transportation Science and Engineering, Beihang University, Beijing, China. His research interests include intelligent transportation systems, urban traffic operation and control, decision making, trajectory planning, and trajectory tracking control of intelligent vehicles. He is a member of the IEEE ITS Society.

**Bin Zhou** received the Ph.D. degree from the School of Transportation Science and Engineering, Beihang University, Beijing, China, in 2018. He is currently an Assistant Professor with the School of Transportation Science and Engineering, Beihang University. His research interests include deep-learning and intelligent vehicle.

**Guizhen Yu** received the Ph.D. degree from Jilin University, China, in 2003. He is currently a Full Professor and a Directory with the School of Transportation Science and Engineering, Beihang University, China. His research interests include intelligent vehicle and urban traffic operations.

Authorized licensed use limited to: American University of Beirut. Downloaded on January 28,2026 at 19:35:21 UTC from IEEE Xplore. Restrictions apply.